

Μεταγλωττιστές 2025-26

Συντακτική ανάλυση LL(1):
Σύνολα FOLLOW

Περαιτέρω χαλάρωση περιορισμών

- ~~Θα πρέπει να μην εμφανίζονται~~ ~~κενές παραγωγές~~ (ϵ)
 - Οι κενές παραγωγές είναι απαραίτητες ή/και βοηθούν στη συγγραφή γραμματικών LL(1)
 - Εμφανίζονται στην απαλοιφή της **αριστερής αναδρομής** και την αφαίρεση του **κοινού παράγοντα** από τους κανόνες
 - Βοηθούν στην αποτύπωση εννοιών όπως «μηδέν ή περισσότερες φορές»
- Μια κενή παραγωγή $A \rightarrow \epsilon$ σημαίνει ότι
 - Αν ακολουθήσουμε τον κανόνα, το A «**εξαφανίζεται**»
 - **Χωρίς να καταναλωθεί** token εισόδου

Σύνολα FIRST και κενές παραγωγές

- Όταν υπάρχουν κενές παραγωγές στη γραμματική, ο υπολογισμός των συνόλων FIRST **τροποποιείται**
- Αν υπάρχει κανόνας $A \rightarrow Bx$ και το **B παράγει ϵ** , τότε το $\text{FIRST}(A)$ δεν περιλαμβάνει μόνο το $\text{FIRST}(B)$!
 - Εφόσον το B μπορεί να “εξαφανιστεί”, στο $\text{FIRST}(A)$ πρέπει να **προσθέσουμε** και το $\text{FIRST}(x)$

Εύρεση συνόλων FIRST

(όταν υπάρχουν κενές παραγωγές)

- Ξεκινάμε με **άδεια** σύνολα FIRST για κάθε μη τερματικό σύμβολο της γραμματικής
- Για κάθε κανόνα της γραμματικής:
 - Αν το δεξιό μέρος του κανόνα ξεκινά με **τερματικό a ή το ϵ** , προσθέτουμε το a **ή το ϵ** στο σύνολο FIRST του μη τερματικού που βρίσκεται στο αριστερό μέρος του κανόνα
 - Αν το δεξιό μέρος του κανόνα ξεκινά με **μη τερματικό A** , προσθέτουμε τα σύμβολα του $FIRST(A)$ που ξέρουμε εκείνη τη στιγμή στο σύνολο FIRST του μη τερματικού που βρίσκεται στο αριστερό μέρος του κανόνα
 - Αν το $FIRST(A)$ περιέχει το ϵ , προσθέτουμε, αντί για το ϵ , το σύνολο $FIRST$ του συμβόλου μετά το A
 - Επαναλαμβάνουμε αν και το επόμενο σύμβολο παράγει ϵ . Αν όλα τα σύμβολα στο δεξιό μέρος του κανόνα παράγουν ϵ , προσθέτουμε το ϵ .
- Επαναλαμβάνουμε τη διαδικασία από την αρχή, έως ότου να μην μπορεί να προστεθεί άλλο σύμβολο σε κάποιο σύνολο FIRST

Παράδειγμα

Σύνολα FIRST	Κανόνες
	$\text{Session} \rightarrow \text{Facts Question}$ $\quad \quad \quad \text{ (Session) Session}$
	$\text{Facts} \rightarrow \text{Fact Facts}$ $\quad \quad \quad \epsilon$
	$\text{Fact} \rightarrow ! \text{ string}$
	$\text{Question} \rightarrow ? \text{ string}$

Παράδειγμα από το: Dick Grune. 2010. *Parsing Techniques: A Practical Guide* (2nd. ed.). Springer

Πρώτο πέρασμα

Σύνολα FIRST	Κανόνες
(	$\text{Session} \rightarrow \text{Facts Question}$ $\quad \mid (\text{Session}) \text{Session}$
ϵ	$\text{Facts} \rightarrow \text{Fact Facts}$ $\quad \mid \epsilon$
!	$\text{Fact} \rightarrow ! \text{string}$
?	$\text{Question} \rightarrow ? \text{string}$

Παράδειγμα από το: Dick Grune. 2010. *Parsing Techniques: A Practical Guide* (2nd. ed.). Springer

Δεύτερο πέρασμα

Από το
FIRST(Question)

Παράγει ε

Συμβολα FIRST	Κανόνες
(?	Session -> Facts Question (Session) Session
ε !	Facts -> Fact Facts ε
!	Fact -> ! string
?	Question -> ? string

Παράδειγμα από το: Dick Grune. 2010. *Parsing Techniques: A Practical Guide* (2nd. ed.). Springer

Τρίτο πέρασμα

Σύνολα FIRST	Κανόνες
(? !	Session -> Facts Question (Session) Session
ε !	Facts -> Fact Facts ε
!	Fact -> ! string
?	Question -> ? string

Παράδειγμα από το: Dick Grune. 2010. *Parsing Techniques: A Practical Guide* (2nd. ed.). Springer

Το πρόβλημα με τις κενές παραγωγές

- Αν υπάρχει κανόνας $A \rightarrow \epsilon$, με **ποιο κριτήριο** επιλέγω αυτόν τον κανόνα;
 - Με τι πρέπει να συγκρίνω το επόμενο token στην είσοδο;
 - **Προσοχή: Χωρίς να καταναλώσω** το επόμενο token εισόδου
 - Υπάρχει κάτι αντίστοιχο με τα σύνολα FIRST;

Σύνολα FOLLOW

- Για να αποφασίσω πότε ακολουθώ μια κενή παραγωγή $A \rightarrow \varepsilon$
- **FOLLOW(A)** είναι το σύνολο **όλων** των τερματικών συμβόλων που πιθανόν ακολουθούν το A, σε οποιαδήποτε παραγωγή
 - Δεν μπορούμε να ξέρουμε ακριβώς τι ακολουθεί το A ανά πάσα στιγμή
 - Εξαρτάται από τους προηγούμενους κανόνες που έχουν χρησιμοποιηθεί ως τώρα
 - Το γνωρίζουμε μόνο κατά την εκτέλεση της συντακτικής ανάλυσης
 - Το σύνολο FOLLOW μπορεί να υπολογιστεί εκ των προτέρων και δίνει μια προσεγγιστική λύση, χωρίς να επηρεάζεται η ορθή λειτουργία του συντακτικού αναλυτή
 - Αν υπάρχει συντακτικό σφάλμα, **ίσως** εκτελεστούν μερικές (κενές) παραγωγές παραπάνω πριν αυτό εντοπιστεί

Εύρεση συνόλων FOLLOW

- Τα σύνολα FOLLOW αρχικά είναι κενά.
- Για κάθε δεξί μέρος των κανόνων της γραμματικής:
 - Για κάθε μη τερματικό B και τυχαίες ακολουθίες συμβόλων α, β σε κανόνες της μορφής $A \rightarrow \alpha B \beta$, προσθέτουμε το $\text{FIRST}(\beta)$ – εκτός από το ϵ – στο $\text{FOLLOW}(B)$.
 - το α μπορεί να μην υπάρχει
 - Για κάθε μη τερματικό B και τυχαία ακολουθία συμβόλων α σε κανόνες της μορφής $A \rightarrow \alpha B$, προσθέτουμε το $\text{FOLLOW}(A)$ στο $\text{FOLLOW}(B)$.
 - Για κάθε μη τερματικό B και τυχαίες ακολουθίες συμβόλων α, β σε κανόνες της μορφής $A \rightarrow \alpha B \beta$, όπου το ϵ ανήκει στο $\text{FIRST}(\beta)$, προσθέτουμε το $\text{FOLLOW}(A)$ στο $\text{FOLLOW}(B)$.
- Επαναλαμβάνουμε ξανά τη διαδικασία για όλους τους κανόνες, έως ότου να μην υπάρχουν νέες προσθήκες στα σύνολα FOLLOW.

Παράδειγμα

Τεχνητό δεξιό μέρος, μπαίνει για να συμπεριλάβουμε το end-of-text (#)

Σύνολα FOLLOW	Σύνολα FIRST	Κανόνες
		<code>-> Session #</code>
		<code>Session -> Facts Question (Session) Session</code>
		<code>Facts -> Fact Facts ε</code>
		<code>Fact -> ! string</code>
		<code>Question -> ? string</code>

Παράδειγμα από το: Dick Grune. 2010. *Parsing Techniques: A Practical Guide* (2nd. ed.). Springer

Παράδειγμα

Σύνολα FOLLOW	Σύνολα FIRST	Κανόνες
		<code>-> Session #</code>
	<code>? !</code> <code>(</code>	<code>Session -> Facts Question</code> <code> (Session) Session</code>
	<code>!</code> <code>ε</code>	<code>Facts -> Fact Facts</code> <code> ε</code>
	<code>!</code>	<code>Fact -> ! string</code>
	<code>?</code>	<code>Question -> ? string</code>

Παράδειγμα από το: Dick Grune. 2010. *Parsing Techniques: A Practical Guide* (2nd. ed.). Springer

Παράδειγμα

Σύνολα FOLLOW	Σύνολα FIRST	Κανόνες
		$\rightarrow$ Session #
#)	? ! (	Session $\rightarrow$ Facts Question (Session) Session
?	! ϵ	Facts $\rightarrow$ Fact Facts ϵ
! ?	!	Fact $\rightarrow$! string
#)	?	Question $\rightarrow$? string

Παράδειγμα από το: Dick Grune. 2010. *Parsing Techniques: A Practical Guide* (2nd. ed.). Springer

Χρήση συνόλων FOLLOW

- Σε κάθε συνάρτηση μη τερματικού συμβόλου με ε στο σύνολο **FIRST** αυτού
- Προσθέτουμε έναν κλάδο if που απλώς επιστρέφει χωρίς να καταναλώνει είσοδο (χωρίς κλήση match())
- Όταν εμφανιστεί κάποιο από τα σύμβολα του συνόλου FOLLOW του μη τερματικού

```
def Facts():  
    # FOLLOW(Facts) = { ? }  
    ...  
  
    elif next_token == '?':  
        return  
    ...
```

Σύνολα FOLLOW και LL(1)

- Εάν υπάρχουν οι εναλλακτικοί κανόνες

$$A \rightarrow x \mid y \mid \dots \mid z$$

- Γνωρίζουμε ήδη ότι θα πρέπει τα $\text{FIRST}(x)$, $\text{FIRST}(y)$, $\dots$, $\text{FIRST}(z)$ να μην έχουν κανένα κοινό στοιχείο μεταξύ τους
- Τι θα πρέπει προσέξουμε όταν χρησιμοποιούμε κενές παραγωγές και σύνολα FOLLOW για να παραμείνει η γραμματική LL(1);
 - Μόνο ένα από τα $x, y, \dots, z$ επιτρέπεται να παράγει ή να είναι ϵ – αλλιώς η γραμματική είναι αμφίσημη, συνεπώς δεν είναι LL(1)
 - Περιγράφεται και ως “FOLLOW-FOLLOW conflict”
 - Έστω ότι το x παράγει ή είναι ϵ . Τότε τα $\text{FIRST}(y)$, $\dots$, $\text{FIRST}(z)$ δεν θα πρέπει να έχουν κοινά στοιχεία με το $\text{FOLLOW}(A)$ – αλλιώς δεν μπορούμε να επιλέξουμε το x έναντι των άλλων με βάση το επόμενο token εισόδου
 - “FIRST-FOLLOW conflict”

“Logo graphics” – Γραμματική με ϵ

CommandList $\rightarrow$ Command CommandList | ϵ

Command $\rightarrow$ Move | Turn | Pen

| repeat num [CommandList]

Move $\rightarrow$ forward Intvalue | backward Intvalue

Turn $\rightarrow$ left Intvalue | right Intvalue

Pen $\rightarrow$ up | down

Intvalue $\rightarrow$ + num | - num | num

```
up
forward -120
right +20
down
repeat 72 [
  repeat 5 [
    forward 100
    left 72
  ]
  right 5
]
backward 600
left -20
```

- Μπορούμε να γράψουμε τη γραμματική με πιο κατανοητό τρόπο
 - Στους σκοπούς της γραμματικής είναι και η **τεκμηρίωση**

“Logo graphics” με ε – Σύνολα FIRST/FOLLOW

Σύνολα FOLLOW	Σύνολα FIRST	Κανόνες Γραμματικής
] #	repeat forward backward left right up down ε	CommandList → Command CommandList ε
] repeat forward backward left right up down #	repeat forward backward left right up down	Command → Move Turn Pen repeat num [CommandList]
<i>όπως το προηγούμενο</i>		Move → forward Intvalue backward Intvalue
<i>όπως το προηγούμενο</i>		Turn → left Intvalue right Intvalue
<i>όπως το προηγούμενο</i>		Pen → up down
<i>όπως το προηγούμενο</i>		Intvalue → + num - num num